算法部分

前缀树（Trie）

```
class Trie:
    def __init__(self):
        self.tree = {}
    def insert(self, word: str) -> None:
        cur = self.tree
        for i in range(len(word)-1):
            a = word[i]
            if a in cur:
                cur = cur[a]
            else:
                cur[a] = {}
                cur = cur[a]
        if word[-1] in cur:
            cur = cur[word[-1]]
            cur[None] = {}
        else:
            cur[word[-1]] = {None:{}}
    def search(self, word: str) -> bool:
        cur = self.tree
        for t in list(word):
            if t not in cur:
                return False
            cur = cur[t]
        if None in cur:
            return True
        return False
    def startsWith(self, prefix: str) -> bool:
        cur = self.tree
        for t in list(prefix):
            if t not in cur:
                return False
            cur = cur[t]
        return True
trie = new Trie()
trie.insert("apple") # {'a': {'p': {'p': {'l': {'e': {None: {}}}}}}}
trie.search("apple");   # True
trie.search("app");     # False
trie.startsWith("app"); # True
```

```
trie.insert("app"); # {'a': {'p': {'p': {'l': {'e': {None: {}}}, None: {}}}}}
trie.search("app");     # True
trie.insert("b") # {'a': {'p': {'p': {'l': {'e': {None: {}}}, None: {}}}}, 'b': {None:
```

线段树（segment tree）

对[1,2,3]建树

6

1 5

1 2 3

```
tree = [0] * (2*n)
def build(arr, n):
    for i in range(n):
        tree[n+i] = arr[i]
    for i in range(n-1, 0, -1):
        # 2*i 是左孩子，2*i + 1 是右孩子
        tree[i] = tree[2*i] + tree[2*i+1]
def updateTreeNode(p, value, n):
    p = p+n
    tree[p] = value
    i = p
    while i > 1:
        i = i//2
        # 父节点等于左右两个子节点之和
        tree[i] = tree[2*i] + tree[2*i+1]
def query(l, r, n): #[l, r)
    res = 0
    l += n
    r += n
    while l < r:
        if (l % 2 != 0):
            res += tree[l]
            l += 1
        if (r % 2 != 0):
            r -= 1
            res += tree[r]
        l = l // 2
        r = r // 2
    return res
```

树状数组（BIT/Fenwick Tree）

```python
# Binary Indexed Tree
class BIT:
    def __init__(self, n):
        self.size = n
        self.tree = [0] * (n + 1)
    def lowbit(self, x):
        return x & -x
    def update(self, i, delta):
        """将第 i 个元素增加 delta"""
        while i <= self.size:
            self.tree[i] += delta
            i += self.lowbit(i)
    def query(self, i):
        """查询前 i 个元素的和"""
        s = 0
        while i > 0:
            s += self.tree[i]
            i -= self.lowbit(i)
        return s
```

树（tree）

所有给你一个表达式让你思考怎么地怎么地的题，先思考答案 **从什么树而来（大多数情况）** 或是什么stack而来，然后在草稿纸上画出树，再思考建树方法
关于建树，一般就是前序，中序，后序，**后缀波兰表达式** 转化，中间一般需要stack辅助（比如后缀波兰表达式建树就利用stack，思路跟求值差不多）
优先队列二叉堆：先建完全二叉树，再更新的时候不停地交换，程序实现时不必用树，用列表即可
脑子卡bug一定记住要用递归
bst插入就是从根开始走
avl插入（ll型根右旋，lr型先左子树左旋再根右旋，rr型根左旋，rl型先右子树右旋再根左旋）
遍历（Traversal）：
深度优先遍历（DFS）：
前序遍历（Preorder）：根 → 左 → 右
中序遍历（Inorder）：左 → 根 → 右（BST 中序遍历结果为有序数组）
后序遍历（Postorder）：左 → 右 → 根
广度优先遍历（BFS）（层序遍历）：按层从左到右依次遍历。

```python
from collections import deque
def cengxubianli(tree):
    result = []
    queue = deque([tree])
    while queue:
        for _ in range(len(queue)):
            node = queue.popleft()
            result.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
    print(*result)
```

找根节点的技巧
根节点 = 出现在 all_nodes 但没出现在 child_nodes 的那个

```python
root = (all_nodes - child_nodes).pop()
```

霍夫曼算法：将具有最小频率的两棵二叉树合并为一棵二叉树
测试树

```python
class TreeNode:
    def __init__(self,val=0,left=None,right=None):
        self.val = val
        self.left = left
        self.right = right
import math
mylist = [1,2,2,None,3,None,3]
cengshu = int(math.log2(len(mylist)+1))-1
def constructtree(ceng,k):#层序，第ceng层的第k个，根在第0层
    if ceng > cengshu:
        return
    m = (1 << ceng)+k-2
    if not mylist[m]:
        return
    node = TreeNode(mylist[m])
    node.left = constructtree(ceng+1,2*k-1)
    node.right = constructtree(ceng+1,2*k)
    return node
result = []
def preorder_traversal(node):
    if node:
        result.append(node.val)
        preorder_traversal(node.left)
        preorder_traversal(node.right)
tree = constructtree(0,1)
preorder_traversal(tree)
print(*result)
```

广度优先搜索(bfs)

一定要注意matrix的小情况和特殊情况，以及起点和终点的情况
queue内可以是多个指标，比如位置+step+魔力值这种
start不一定只有一个，可以多个同时出发

```python
from collections import deque
def bfs(start):
    q = deque()
    q.append((start, 0))
    visited = {start}     # 必须用set，否则会mle
    prev = {start: None}
    while q: # 强迫自己必须只能套一步while
        cur, step = q.popleft()
        if is_target(cur):
            path = []
            node = cur
            while node is not None:
                path.append(node)
                node = prev[node]
            path.reverse()
            print("最短路径:", path)
            return step
        for nxt in get_neighbors(cur):
            if nxt not in visited:
                visited.add(nxt)
                prev[nxt] = cur
                q.append((nxt, step+1))
    return -1   # 找不到
```

双向bfs（可以从 end 反向扩展，反向扩展的规则和正向一样简单）

```python
from collections import deque
def bidirectional_bfs(start, target, get_neighbors):
    # 若起点和终点相同，直接返回
    if start == target:
        return 0
    # 前向 & 后向队列
    q1 = deque([start])
    q2 = deque([target])
    # 前后访问集合
    visited1 = {start}
    visited2 = {target}
    steps = 0
    while q1 and q2:
        # —— 优化点：永远扩展较小的那个队列 —— #
        if len(q1) > len(q2):
            q1, q2 = q2, q1
            visited1, visited2 = visited2, visited1
        steps += 1
        size = len(q1)
        for _ in range(size):
            node = q1.popleft()
            for nei in get_neighbors(node):
                # 1. 如果在对方 visited 中，说明相遇了
                if nei in visited2:
                    return steps
                # 2. 如果没访问过，加入队列
                if nei not in visited1:
                    visited1.add(nei)
                    q1.append(nei)
    return -1   # 没有路径
```

并查集（Disjoint Set）

记得编号是否需要减一与index统一

Find（path compression）每次操作后都把沿途的每个节点直接挂到根上

```python
Parent = [i for i in range(n)]
def find(i):
    if (Parent[i] == i):
        return i
    else:
        result = find(Parent[i])
        Parent[i] = result
        return result


rank = [1]*n
def union(i,j):
    i1 = find(i)
    j1 = find(j)
    if i1 == j1:
        return
    if rank[i1] < rank[j1]:
        Parent[i1] = j1
    elif rank[i1] > rank[j1]:
        Parent[j1] = i1
    else:
        Parent[j1] = i1
        rank[i1] += 1


Size = [1]*n
def union(i, j):
    i1 = find(i)
    j1 = find(j)
    if i1 == j1:
        return
    if Size[i1] < Size[j1]:
        Parent[i1] = j1
        Size[j1] += Size[i1]
    else:
        Parent[j1] = i1
        Size[i1] += Size[j1]
```

栈（Stack）

中缀转后缀算法

数字 扔进输出。

左括号 入栈。
操作符 把栈里比自己同等级或强的都弹出去，然后自己入栈。
右括号 栈内“大清扫”直到遇见左括号。
最后 把栈里剩下的所有东西全部倒出来。

链表（Linked List）

链表反转

```
def reverse_linked_list(head: ListNode) -> ListNode:
    prev = None
    curr = head
    while curr is not None:
        prev = ListNode(curr.val,prev)
        curr = curr.next
    return prev


def reverse_linked_list(head: ListNode) -> ListNode:
    prev = None
    curr = head
    while curr is not None:
        next_node = curr.next  # 暂存当前节点的下一个节点
        curr.next = prev       # 将当前节点的下一个节点指向前一个节点
        prev = curr            # 前一个节点变为当前节点
        curr = next_node       # 当前节点变更为原先的下一个节点
    return prev
```

合并两个排序链表

```python
def merge_sorted_lists(l1, l2):
    dummy = Node(0) #dummy（哑节点 / 哨兵节点）能避免处理链表头节点为空的边界情况
    tail = dummy
    while l1 and l2:
        if l1.data < l2.data:
            tail.next = l1
            l1 = l1.next
        else:
            tail.next = l2
            l2 = l2.next
        tail = tail.next
    if l1:
        tail.next = l1
    else:
        tail.next = l2
    return dummy.next
```

查找链表的中间节点（快慢指针）

```python
def find_middle_node(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    return slow
```

1->1;2->2;3->2;4->3;5->3;n->(n//2)+1
双向链表

```python
class Node:
    def __init__(self, data):
        self.data = data  # 节点数据
        self.next = None  # 指向下一个节点
        self.prev = None  # 指向前一个节点
```

循环链表可以用快慢指针去找圈
测试链表

```python
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next
def construct(mylist):
    head = ListNode()
    cur = head
    for num in mylist:
        cur.next = ListNode(num)
        cur = cur.next
    return head.next
def show(head):
    result = []
    while head:
        result.append(head.val)
        head = head.next
    return result
```

语法部分

Python字符串提供的方法

Method Name	Use	Explanation
`center`	`astring.center(w)`	Returns a string centered in a field of size `w`
`ljust`	`astring.ljust(w)`	Returns a string left-justified in a field of size `w`
`rjust`	`astring.rjust(w)`	Returns a string right-justified in a field of size `w`
`find`	`astring.find(item)`	Returns the index of the first occurrence of `item`

Operation Name	Operator	Explanation
`|`	`aset | otherset`	Returns a new set with all elements from both sets
`&`	`aset & otherset`	Returns a new set with only those elements common to both sets

Operation Name	Operator	Explanation
`-`	`aset - otherset`	Returns a new set with all items from the first set not in second
`<=`	`aset <= otherset`	Asks whether all elements of the first set are in the second
`union`	`aset.union(otherset)`	Returns a new set with all elements from both sets

random模块

函数	功能	返回类型	取值范围 / 特点	常见用途
`random()`	生成随机小数	`float`	`[0.0, 1.0)`	概率判断
`randint(a, b)`	随机整数	`int`	`a ≤ x ≤ b`	随机页数
`randrange(a, b)`	随机整数	`int`	`a ≤ x < b`	时间/事件模拟
`choice(seq)`	随机选一个元素	任意	序列中任选	随机位置
`choices(seq, k=n)`	可重复抽样	`list`	允许重复	概率抽样
`sample(seq, k)`	不重复抽样	`list`	无重复	抽签/组合
`shuffle(seq)`	原地打乱	`None`	修改原列表	洗牌
`uniform(a, b)`	随机浮点数	`float`	`[a, b)`	连续分布

运算符计算

```
opers = {'+':operator.add, '-':operator.sub, '*':operator.mul, '/':operator.truediv}
opers[运算符](a,b)
```

对类用heap的话，要利用__lt__定义大小比较关系

```
def __lt__(self, other):
    if self.weight == other.weight:
        return self.char < other.char
    return self.weight < other.weight
```